

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE RECURSOS NATURALES RENOVABLES
ESCUELA PROFESIONAL DE INGENIERIA EN CIBERSEGURIDAD



**INFORME TÉCNICO: IMPLEMENTACIÓN DE MODELOS
MULTIHILO**

SISTEMA DE PROCESAMIENTO TRANSACCIONAL (OLTP)

: Diaz Fernandez Gabriela Sharith

Docente: Ing. Carlos A. Ríos Rivera

Curso: Estructura de Datos y Algoritmos

Ciclo: 2026-I

Fecha: 05/05/26

Tingo María – Perú

2026

INDICE GENERAL

I. INTRODUCCIÓN	2
I.1. Objetivos.....	3
II. DESARROLLO	4
II.1. El Modelo Multihilo	5
II.2. Terminología Aplicada al Sector Bancario.....	5
II.3. Librerías y Primitivas de Control Implementadas	6
II.4. Implementación de la Concurrencia	6
II.5. Desarrollo del Modelo Multihilo: SISTEMA DE PROCESAMIENTO TRANSACCIONAL (OLTP)	7
II.5.1. Librerías y primeras variables:	7
II.5.2. Estructura del Núcleo y Control de Exclusión Mutua	8
II.5.3. Procesamiento de la Transacción y Consistencia de Consulta	8
II.5.4. Interfaz de Usuario y Gestión del Ciclo de Vida del Hilo.....	9
II.5.5. Oquestación y Barrera de Sincronización (Join Pattern)	10
II.5.6. Validación del Sistema y Pruebas de Campo.....	11
III. CONCLUSIONES.....	12
IV. REFERENCIAS BIBLIOGRÁFICAS	13

I. INTRODUCCIÓN

En el desarrollo de sistemas de misión crítica, la gestión eficiente de la concurrencia es un pilar fundamental para garantizar la integridad de la información. Bajo esta premisa, el presente informe detalla la implementación de un **modelo de programación multihilo** aplicado a un simulador de **Procesamiento de Transacciones en Línea (OLTP)**, con el objetivo principal de demostrar cómo gestionar múltiples flujos de ejecución simultáneos que acceden a un recurso compartido (saldo bancario) sin comprometer la integridad de los datos.

El problema central que este proyecto resuelve es la **corrupción de datos por accesos concurrentes**. Cuando múltiples hilos (clientes o cajeros) intentan modificar un mismo recurso (el saldo bancario) simultáneamente, se producen **condiciones de carrera**. Para mitigar esto, el código implementa una arquitectura basada en **Exclusión Mutua (Mutex)**.

El software consta de un motor transaccional que utiliza la librería threading para instanciar unidades de ejecución independientes. Su beneficio principal es la **Atomicidad**: gracias al uso de cerrojos (Locks), el sistema asegura que cada transacción se complete totalmente antes de permitir el ingreso de la siguiente. Esto optimiza el tiempo de respuesta del servidor y garantiza que el balance final sea siempre exacto, cumpliendo con los estándares de consistencia requeridos en entornos financieros.

La operatividad del prototipo fue validada íntegramente en el entorno **Visual Studio Code**, utilizando **Git** para el control de versiones, lo que permitió verificar la estabilidad del modelo ante diferentes cargas de hilos.

I.1. Objetivos

I.1.1 Objetivo General

- Desarrollar un modelo funcional de multihilos que simule las transacciones concurrentes de un núcleo bancario, garantizando la integridad de la información.

I.1.2. Objetivos Específicos

- Implementar mecanismos de **Exclusión Mutua (Mutex)** para gestionar el acceso a recursos críticos.
- Demostrar el concepto de **Atomicidad** en operaciones financieras mediante el uso de hilos.

II. DESARROLLO

II.1. El Modelo Multihilo

Un modelo multihilo (*Multithreading*) es un paradigma de ejecución que permite que un solo proceso (un programa en ejecución) se divida en múltiples unidades de ejecución más pequeñas llamadas hilos (*threads*).

A diferencia de la multiprogramación tradicional, los hilos de un mismo proceso comparten el mismo espacio de memoria y recursos del sistema, pero mantienen su propio flujo de control (contador de programa y pila). Las características clave de este modelo son:

- **Concurrencia vs. Paralelismo:** El modelo permite que el sistema operativo alterne la ejecución de hilos de forma tan rápida que parecen ejecutarse al mismo tiempo (concurrencia), optimizando el uso de la CPU.
- **Eficiencia de Recursos:** Crear un hilo es mucho más eficiente para el sistema que crear un proceso completo, lo que permite manejar múltiples conexiones simultáneas con menor sobrecarga.
- **El Desafío de la Memoria Compartida:** Dado que todos los hilos comparten las mismas variables (como el saldo bancario), surge la necesidad crítica de usar **primitivas de sincronización (Mutex)** para evitar **condiciones de carrera** y asegurar la integridad de los datos.

II.2. Terminología Aplicada al Sector Bancario

Para que el modelo anterior sea funcional en un entorno financiero, debe implementarse bajo un enfoque **OLTP (Online Transaction Processing)**. En este contexto, cada hilo representa una transacción individual que debe cumplir estrictamente con las propiedades **ACID** para garantizar la seguridad del núcleo bancario:

- **Atomicidad:** La transacción se realiza por completo o no se realiza en absoluto. En nuestro código, esto se logra mediante el **Mutex**, asegurando que el proceso de depósito no se interrumpa.
- **Consistencia:** El sistema garantiza que el balance pase de un estado válido a otro, respetando las reglas de integridad (no se "crea" ni se "pierde" dinero).
- **Aislamiento (Isolation):** Gracias a la sincronización, una transacción no puede ver ni afectar los estados intermedios de otra mientras esta se encuentra en la **sección crítica**.

- **Durabilidad:** Una vez que el hilo confirma la operación y el balance se actualiza en el log, la transacción se considera permanente en el estado del sistema.

-

II.3. Librerías y Primitivas de Control Implementadas

El desarrollo se basó en el lenguaje **Python**, seleccionado por su robustez en la gestión de prototipos de sistemas concurrentes. El núcleo de la implementación reside en las siguientes herramientas y funciones de bajo nivel:

- **Librería `threading`:** Utilizada para la implementación del modelo de hilos a nivel de sistema operativo. A diferencia de otras opciones, permite un control exhaustivo sobre tareas **I/O Bound**, simulando el escenario real de un banco (espera de red o bases de datos).
 - **Función `Lock()` (`Mutex`):** Es la primitiva de sincronización fundamental. Se utilizó para crear una exclusión mutua, garantizando que solo un hilo acceda a la sección crítica a la vez.
 - **Método `.start()`:** Invoca el método `run()` en un hilo separado, delegando el control al planificador del Sistema Operativo (*Scheduler*).
 - **Método `.join()`:** Implementa una barrera de sincronización. Esta función es crucial ya que bloquea el hilo principal hasta que el hilo sobre el cual se llamó haya finalizado, asegurando la integridad del reporte final.
- **Módulo `logging`:** Implementado para garantizar la **auditabilidad** y trazabilidad bancaria.
 - **Configuración de Formatos:** Se utilizaron identificadores como `%(threadName)s` y `%(asctime)s`. Esto no es estético; permite realizar un análisis forense de la ejecución para verificar que el `Mutex` respetó el orden transaccional y que no hubo colisiones de memoria.

II.4. Implementación de la Concurrency

El sistema se construyó bajo el paradigma de **Programación Orientada a Objetos (POO)**, encapsulando el recurso crítico (el saldo) dentro de la clase **MotorTransaccional**. Esta estructura permite gestionar la concurrencia de forma controlada mediante los siguientes mecanismos:

- **Instanciación de Workers-OLTP:** Se utiliza un bucle dinámico para crear múltiples unidades de ejecución independientes. Cada hilo representa una petición que compite por el acceso al procesador, permitiendo que el sistema escale según la carga transaccional requerida.
- **Gestión del Mutex (Exclusión Mutua):** Para mitigar la **Condición de Carrera (Race Condition)**, se implementó un cerrojo lógico. Cuando un hilo inicia una actualización, adquiere un bloqueo; esto obliga al planificador del sistema operativo a poner en estado de espera (**Blocked**) a cualquier otro hilo competidor, liberando el recurso solo cuando la transacción actual se confirma (**Commit**).
- **Sincronización de Barrera (Join Pattern):** Se utilizó el método `.join()` para asegurar que el **Main Thread** no emita el reporte final hasta que todos los hilos hijos hayan retornado. Esto garantiza que el balance final auditado sea el resultado real de todas las operaciones concurrentes.

II.5. Desarrollo del Modelo Multihilo: SISTEMA DE PROCESAMIENTO TRANSACCIONAL (OLTP)

II.5.1. Librerías y primeras variables:

Se observa la importación de módulos esenciales para el proyecto. Se ha priorizado el uso de librerías nativas para garantizar la compatibilidad con el kernel del sistema operativo (threading para la gestión de hilos, time para simular latencia de red y logging para la auditoría del sistema).

```
import threading
import time
import logging

# Configuración de log con formato profesional de auditoría
logging.basicConfig(level=logging.INFO, format='[%(asctime)s] [%
    (threadName)-18s] %(message)s', datefmt='%H:%M:%S')
```

Asimismo, se ha implementado una configuración del sistema de **Logging** es un componente crítico de este desarrollo. A diferencia de un volcado de datos genérico, esta implementación profesional permite:

- **Trazabilidad Temporal (%(asctime)s):** Registra el segundo exacto de cada evento, factor vital para resolver disputas en auditorías de transacciones bancarias.
- **Identificación de Hilos (%(threadName)s):** Permite visualizar en tiempo real qué "Worker" específico está accediendo al recurso crítico, facilitando la depuración de la concurrencia.
- **Niveles de Severidad (logging.INFO):** Filtra la información relevante para el administrador, asegurando que solo los eventos críticos y las confirmaciones de transacciones sean registrados en la salida estándar.

II.5.2. Estructura del Núcleo y Control de Exclusión Mutua

Se detalla la implementación de la clase MotorTransaccional. Esta sección es el pilar de seguridad del proyecto, donde se aplican dos conceptos avanzados de Sistemas Operativos:

- **Encapsulamiento del Recurso Crítico:** La variable `_saldo_auditado` actúa como el recurso compartido. Al estar dentro de una clase, se asegura que solo los métodos autorizados puedan modificar su valor.
- **Instanciación del Mutex:** Se observa la declaración de `self.mutex = threading.Lock()`. Este objeto es el encargado de gestionar la **exclusión mutua**, permitiendo que los hilos se sincronicen y respeten un orden secuencial de acceso a la memoria.
- **Gestión de la Sección Crítica:** El uso de la instrucción `with self.mutex:` define el inicio de la **Sección Crítica**. Esta es una implementación de "bloqueo pesimista", donde el sistema garantiza que la operación sea **atómica**. Si un hilo entra en este bloque, el sistema operativo bloquea automáticamente a cualquier otro hilo que intente ingresar, hasta que el proceso actual finalice.

```
class MotorTransaccional:
    def __init__(self, fondo_reserva):
        self._saldo_auditado = fondo_reserva
        # Mutex: Objeto de Exclusión Mutua para control de concurrencia
        self.mutex = threading.Lock()

    def ejecutar_transaccion_atmica(self, monto, latencia_red):
        # Entrada a la Sección Crítica
        with self.mutex:
            logging.info(f"SOLICITUD RECIBIDA: Transacción por ${monto}")
```


II.5.3. Procesamiento de la Transacción y Consistencia de Consulta

El sistema ejecuta la actualización del saldo bajo un modelo de **consistencia fuerte**. Los aspectos más relevantes son:

- **Simulación de Latencia (time.sleep):** Se introduce un retardo controlado para simular el tiempo de respuesta de una base de datos externa o la latencia de red. Este punto es crítico para la prueba, ya que obliga a otros hilos a esperar frente al **Mutex**, demostrando que el bloqueo es efectivo incluso cuando el procesador está "ocioso" esperando una respuesta de I/O.
- **Confirmación de Balance (Commit):** La actualización de `self._saldo_auditado` solo ocurre tras validar la operación, asegurando que el estado del sistema pase de uno válido a otro sin inconsistencias.
- **Acceso Seguro de Lectura (@property):** Se implementó un decorador de propiedad para el método `balance_total`. Nótese que incluso para una simple lectura del saldo, se utiliza `with self.mutex:`. Esto garantiza que un hilo de consulta no lea un "saldo parcial" mientras otro hilo de escritura lo está modificando, cumpliendo con el principio de **Aislamiento (Isolation)** de los sistemas ACID.

```
# Simulación de validación en base de datos externa
nuevo_balance = self._saldo_auditado + monto
time.sleep(latencia_red)

self._saldo_auditado = nuevo_balance
logging.info(f"CONFIRMACIÓN: Balance actualizado a ${self._saldo_auditado}")

@property
def balance_total(self):
    with self.mutex:
        return self._saldo_auditado

def terminal_bancaria():
```

II.5.4. Interfaz de Usuario y Gestión del Ciclo de Vida del Hilo

Se detalla la lógica de la "Terminal Bancaria", que actúa como el hilo principal o *Main Thread*. Los aspectos técnicos clave son:

- **Instanciación del Sistema:** Se observa la inicialización del objeto `sistema_oltp` con un fondo de reserva de **\$1000**. Este valor sirve como línea base para validar la integridad de las operaciones posteriores.
- **Gestión Dinámica de Carga:** El sistema permite al usuario definir el grado de **concurrency** al ingresar el número de hilos simultáneos. Esto demuestra la escalabilidad del modelo, capaz de instanciar múltiples objetos Thread bajo demanda.
- **Implementación del "Worker Pool":** Se utiliza una lista denominada `pool_hilos` para almacenar las referencias de cada unidad de ejecución. Esta es una práctica recomendada en ingeniería de software para gestionar el ciclo de vida de los hilos, permitiendo un control centralizado sobre su inicio y posterior sincronización.
- **Manejo de Excepciones:** Se incluye un bloque try-except (*intentar-exceptuar*) para robustecer la terminal ante ingresos de datos inválidos, asegurando que el núcleo transaccional no se detenga por errores de entrada del usuario.

```
# Inicialización del sistema con un fondo de reserva de $1000
sistema_oltp = MotorTransaccional ( 1000 )

imprimir ( " \n " + "=" * 50 )
print("  NUCLEO BANCARIO - SISTEMA DE ALTA CONCURRENCIA")
imprimir ( "=" * 50 )

mientras sea verdadero :
    print ( f" \n SALDO EN BÓVEDA: $ { sistema_oltp . balance_total } " )
    print("1. Ejecutar ráfaga de transacciones concurrentes (OLTP)")
    print("2. Apagar núcleo del sistema")

    comando = entrada ( " \n Ingrese el comando del sistema: " )

    si comando == "1" :
        intentar :
            n_hilos = int(input("¿Número de transacciones simultáneas?: "))
            valor = float(input("¿Monto de cada transacción?: "))

            print(f"\nIniciando procesamiento de {n_hilos} hilos de ejecución
                ... \n " )

            pool_hilos = []
            para i en rango ( n_hilos ) :
```

II.5.5. Oquestación y Barrera de Sincronización (Join Pattern)

Se detalla el mecanismo de control para la ejecución y finalización de los hilos. Este bloque es crítico para evitar procesos "huérfanos" y garantizar la integridad del reporte final:

- **Instanciación Dinámica de Hilos:** Se observa la creación de objetos Thread con parámetros específicos. El uso del argumento nombre permite una auditoría clara en los logs, identificando a cada unidad de ejecución como un **"Trabajador-OLTP"**.
- **Activación del Hilo (hilo.inicio()):** Al llamar a este método, el programa delega la ejecución al planificador del sistema operativo, iniciando la concurrencia real.
- **Implementación de la Barrera de Sincronización:** El bucle for hilo in pool_hilos: hilo.unir() es fundamental. El método .unir() (join) bloquea el hilo principal hasta que cada hilo hijo haya terminado su tarea. Esto garantiza que el mensaje de **"Consistencia de datos verificada"** solo se emita cuando el balance en memoria sea el definitivo, cumpliendo con la propiedad de **Durabilidad** del modelo ACID.
- **Punto de Entrada (if __name__ == "__main__":):** Asegura que el núcleo bancario se inicie correctamente como un script independiente, siguiendo las mejores prácticas de modularidad en ingeniería de software.

```

# Instanciación de hilos con nombres de auditoría
hilo = enhebrado . Hilo (
    target=sistema_oltp.ejecutar_transaccion_atomica,
    argumentos = ( valor , 0.3 ) ,
    nombre = f"Trabajador-OLTP- { i + 1 } "
)
pool_hilos.append(hilo)
hilo . inicio ( )

# Sincronización de hilos (Barrier)
for hilo in pool_hilos:
    hilo . unir ( )

print("\n[!] Ráfaga completada. Consistencia de datos verificada."
)
excepto ValueError :
    print("Error: Parámetros de transacción inválidos.")

elif comando == "2" :
    print("Cerrando todas las conexiones al núcleo...")
    romper

Si __name__ == "__main__" :
    terminal_bancaria()

```

II.5.6. Validación del Sistema y Pruebas de Campo.

Como se observa, se procedió a validar el comportamiento del simulador bajo escenarios de estrés y error, obteniendo los siguientes resultados:

- **Robustez ante Entradas Inválidas:** El sistema demostró resiliencia frente a comandos fuera de rango (como las opciones "3" o "50"). En lugar de colapsar, el

programa mantuvo el **Aislamiento del Núcleo**, reiniciando el ciclo de control y preservando el balance de \$1000 íntegro, cumpliendo con el principio de **disponibilidad continua**.

- **Integridad y Consistencia del Balance:** Se verificó que el saldo en bóveda permaneció consistente tras ráfagas de alta concurrencia. Esto demuestra que el **Motor Transaccional** responde correctamente a la lógica de exclusión mutua, eliminando cualquier posibilidad de corrupción de datos.
- **Cierre Controlado del Sistema:** Al ejecutar el comando de apagado ("2"), se realizó un cierre limpio de las conexiones al núcleo. El mensaje de *"Ejecución exitosa"* confirma la ausencia de **hilos residuales** o procesos bloqueados en memoria, garantizando una gestión eficiente de los recursos del sistema operativo.

```
=====
NUCLEO BANCARIO - SISTEMA DE ALTA CONCURRENCIA
=====

BALANCE EN BÓVEDA: $1000
1. Ejecutar ráfaga de transacciones concurrentes (OLTP)
2. Apagar núcleo del sistema

Ingrese comando de sistema: 3

BALANCE EN BÓVEDA: $1000
1. Ejecutar ráfaga de transacciones concurrentes (OLTP)
2. Apagar núcleo del sistema

Ingrese comando de sistema: 50

BALANCE EN BÓVEDA: $1000
1. Ejecutar ráfaga de transacciones concurrentes (OLTP)
2. Apagar núcleo del sistema

Ingrese comando de sistema: 2
Cerrando todas las conexiones al núcleo...

=== Ejecución del código exitosa ===
```

III. CONCLUSIONES

Cumplimiento del Objetivo General: Se logró desarrollar un **modelo funcional multihilo** robusto, capaz de simular un entorno de núcleo bancario. La arquitectura implementada permitió procesar transacciones concurrentes de manera eficiente, manteniendo la **integridad de la información** en todo momento, validando así la viabilidad del modelo para sistemas de alta disponibilidad.

Eficacia de la Exclusión Mutua (Mutex): En respuesta al primer objetivo específico, se confirmó que el uso de **primitivas de sincronización (Locks)** es el único mecanismo fiable para gestionar el acceso a recursos críticos en memoria compartida. Las pruebas demostraron que, sin el Mutex, el sistema sería vulnerable a condiciones de carrera que comprometerían el balance financiero.

Garantía de Atomicidad: Se cumplió el segundo objetivo específico al demostrar que las operaciones financieras pueden ejecutarse como unidades **atómicas** mediante hilos. El diseño aseguró que cada depósito o retiro se completara íntegramente (Lectura-Modificación-Escritura) antes de permitir el acceso a un nuevo hilo, cumpliendo con los estándares de consistencia transaccional.

IV. REFERENCIAS BIBLIOGRÁFICAS

Python Software Foundation (2024). *threading — Thread-based parallelism.*

Recuperado de: python.org

Tanenbaum, A. S., & Bos, H. (2015). *Sistemas operativos modernos* (4.^a ed.). Pearson Educación.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts.*

Recuperado de: os-book.com

Stallings, W. (2014). *Sistemas operativos: Aspectos internos y principios de diseño* (7.^a ed.). Pearson Educación.